

Linked Lists

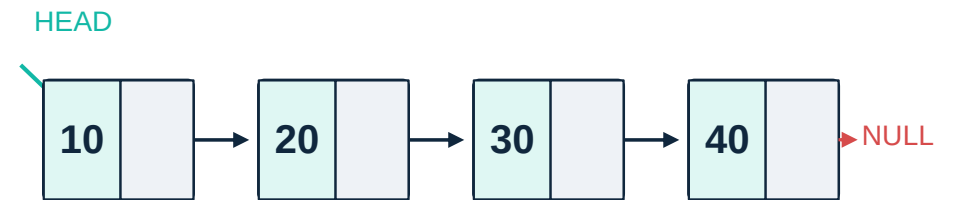
From memory layout to insertion, deletion, circular and doubly linked lists

TEACHING DECK

Concept → Diagram → Algorithm → C code → Practice

Professional teaching edition

A linked list stores nodes anywhere in memory — links preserve the logical order.



Each node = DATA + NEXT pointer

Why do we need linked storage?

A linked list trades direct indexing for flexible memory allocation and easy structural changes.

Problem with sequential allocation

The final size may be unknown. Growing a fixed array may require a new block and data copying.

Frequent insertions / deletions

Shifting many array elements can be expensive. A linked structure changes links instead of shifting data.

Idea: link addressing

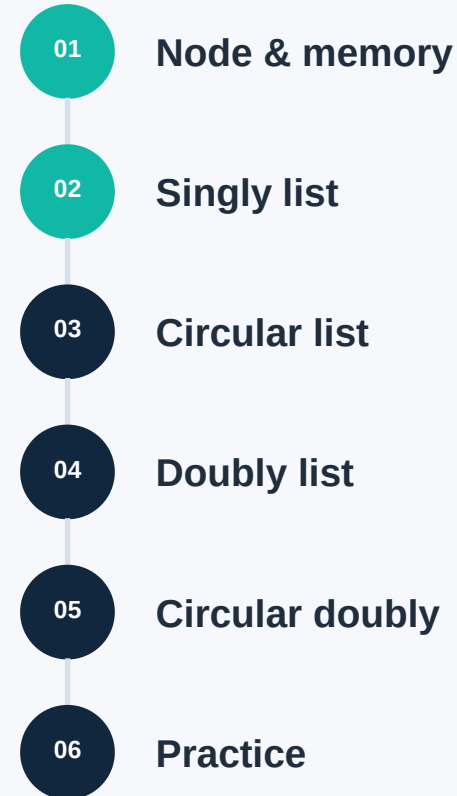
Each node stores data and the address of the next node, creating a one-way chain.

Key trade-off

Linked lists are excellent for dynamic growth and link-based updates, but they do not support $O(1)$ random access like arrays.

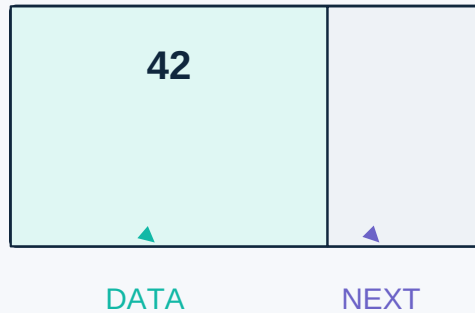
Learning outcomes & lesson map

- Explain how nodes and pointers represent a linked list in memory.
- Trace traversal and search using a temporary pointer.
- Implement insertion and deletion safely in singly, circular, doubly and circular-doubly lists.
- Compare variants using time complexity, pointer overhead and use cases.
- Avoid common C pointer and memory-management errors.



Anatomy of a linked-list node

A node is a user-defined structure containing data and one or more links.



Node definition

```
struct node {  
    int data;           // value stored in the node  
    struct node *next; // address of the next node  
};
```

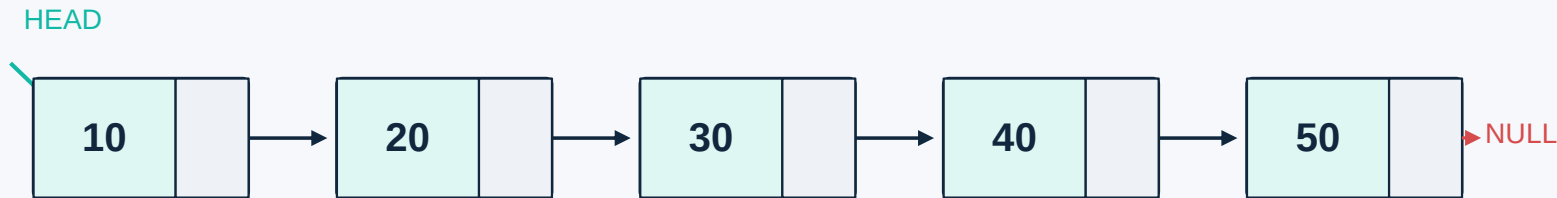
IMPORTANT

The pointer stores an address — not the next data value.
The last node of a normal singly list stores NULL.

MEMORY MODEL

Nodes need not be adjacent in RAM. Links create the logical sequence.

Singly linked list: how the chain works



HEAD stores the address of the first node

Each NEXT points forward

Last NEXT = NULL

Empty list

head == NULL

Logical order

Follow NEXT from HEAD

Random access

No direct index; traverse to reach a node

Core linked-list operations

Most exams and lab programs revolve around six operation families.

Create

Allocate & connect nodes

Traverse

Visit every node

Search

Locate a key value

Insert

Beginning • end • after key

Delete

Beginning • end • after key

Sort

Reorder data / links

TEACHING FOCUS

For each operation, identify exactly which pointer(s) change and in what order.

Creating nodes dynamically with malloc()

Dynamic allocation is the usual approach for linked lists in C.

- 1 Declare a pointer that can point to struct node.
- 2 Request enough heap memory for one complete node.
- 3 Check whether allocation succeeded.
- 4 Fill the data field and initialize the link.
- 5 free() the node when it is removed.

Safe allocation pattern

```
struct node *np;           // 1) node pointer
np = malloc(sizeof *np);    // 2) allocate one
                             // node

if (np == NULL) {          // 3) allocation
    failed
    printf("Out of memory\n");
    return;
}

np->data = 10;              // 4) store data
np->next = NULL;            // no next node yet
```

NOTE

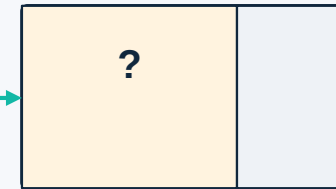
In C, the explicit cast on malloc() is not required. `malloc(sizeof \*np)` is concise and less error-prone.

Create one node: two-line version with comments

Minimal code

```
struct node *np;  
// Declare a pointer that can store the address of a node.  
  
np = malloc(sizeof *np);  
// Allocate enough heap memory for one complete node.
```

np



AFTER malloc

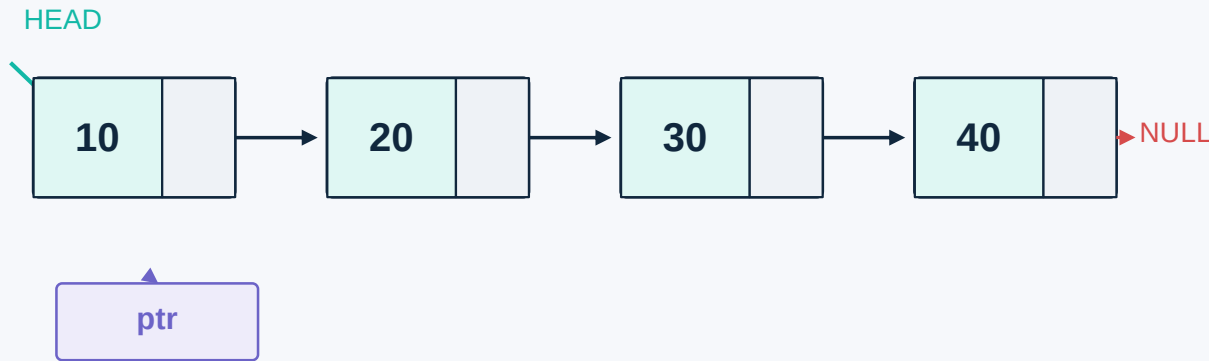
`np` contains the starting address of the newly allocated node.

NEXT STEP

Assign np->data, set np->next, and connect the node to the list.

Traversal: visiting every node once

Use a temporary pointer so HEAD itself is not lost.



- 1 Initialize: $\text{ptr} = \text{head}$
- 2 Process $\text{ptr} \rightarrow \text{data}$
- 3 Move: $\text{ptr} = \text{ptr} \rightarrow \text{next}$
- 4 Stop when $\text{ptr} == \text{NULL}$

INVARIANT

At each iteration, `ptr` points to the node currently being processed.

Traversal algorithm + C function

- 1 SET ptr = head
- 2 WHILE ptr != NULL
- 3 Process ptr->data
- 4 SET ptr = ptr->next
- 5 End when ptr becomes NULL

Traversal in C

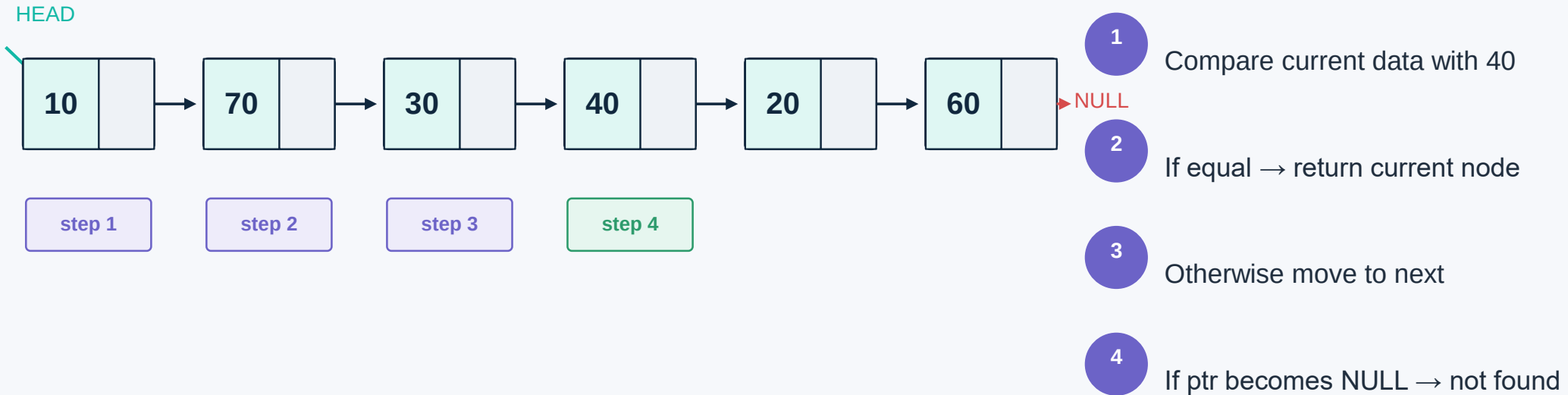
```
void display(struct node *ptr) {  
    while (ptr != NULL) {  
        printf("%d ", ptr->data); // process current node  
        ptr = ptr->next;          // move to next node  
    }  
}  
  
// call  
display(head);
```

COMPLEXITY

Time: $O(n)$ • Extra space: $O(1)$

Searching for key = 40

Search is traversal with a stopping condition.



TRACE RESULT

10 X → 70 X → 30 X → 40 ✓ The matching node address is returned.

Search algorithm + C function

Search in C

```
struct node *search(struct node *ptr, int key) {  
    while (ptr != NULL && ptr->data != key) {  
        ptr = ptr->next;  
    }  
    return ptr; // node address if found, otherwise NULL  
}  
  
struct node *pos = search(head, 40);
```

Found

Return a non-NULL node pointer.

Not found

Return NULL after reaching the end.

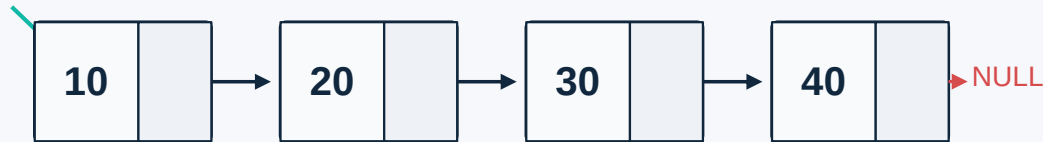
COMPLEXITY

Worst-case time $O(n)$; extra space $O(1)$.

Insertion at the beginning

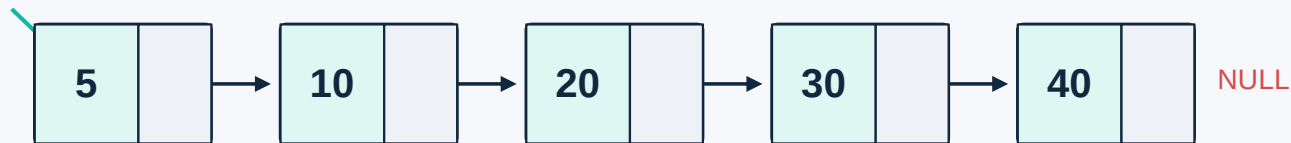
Two pointer assignments are enough — no traversal required.

Before HEAD



1 new->next = head

After HEAD



2 head = new

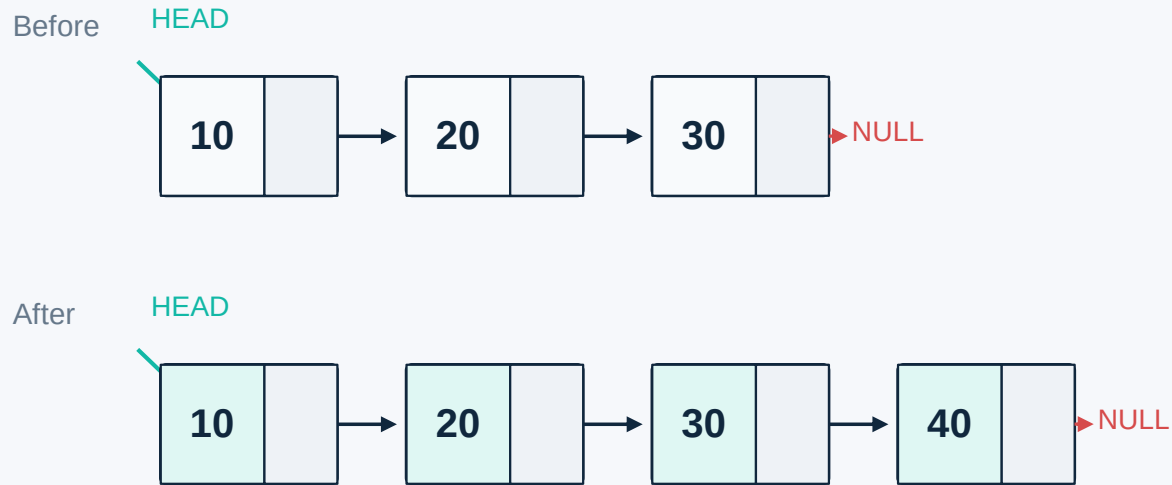
C implementation

```
void push_front(struct node **head, int value) {
    struct node *new = malloc(sizeof *new);
    if (new == NULL) return;

    new->data = value;
    new->next = *head; // old first node becomes
second
    *head = new;      // new node becomes first
}
```

Insertion at the end

Find the last node, then connect its NEXT to the new node.



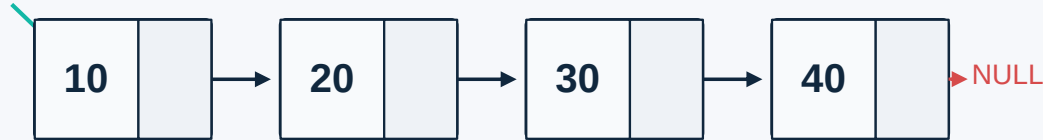
- 1 Create new; new->next = NULL
- 2 If list empty $\rightarrow$ head = new
- 3 Traverse until $\text{ptr} \rightarrow \text{next} == \text{NULL}$
- 4 Set $\text{ptr} \rightarrow \text{next} = \text{new}$

COMPLEXITY

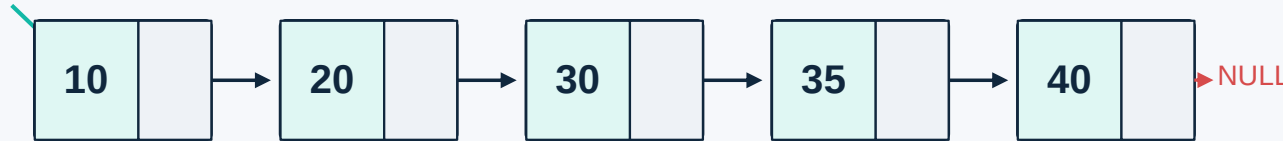
Without a tail pointer: $O(n)$. With a maintained tail: $O(1)$.

Insertion after a node with key = 30

Before HEAD



After HEAD



1 Search for node containing 30

2 $\text{new} \rightarrow \text{next} = \text{ptr} \rightarrow \text{next}$

3 $\text{ptr} \rightarrow \text{next} = \text{new}$

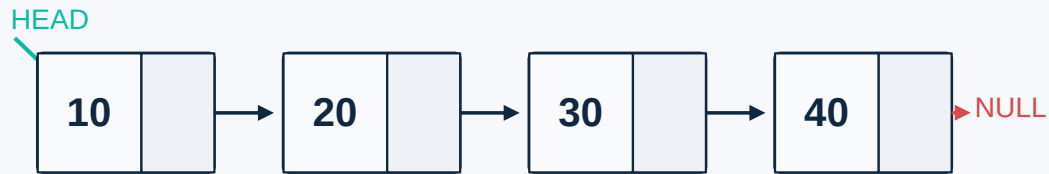
EDGE CASE

If the key is absent, do not insert. If ptr is the last node, $\text{new} \rightarrow \text{next}$ naturally becomes NULL.

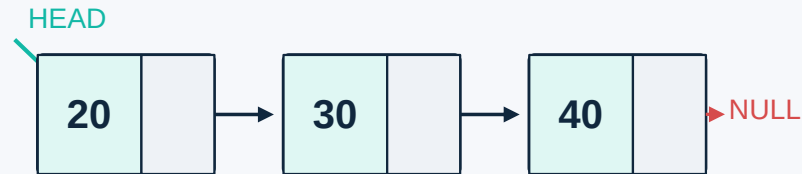
Deleting the first node

Save the old head, advance HEAD, then free the old node.

Before



After



1

If head == NULL → underflow

2

temp = head

3

head = head->next

4

free(temp)

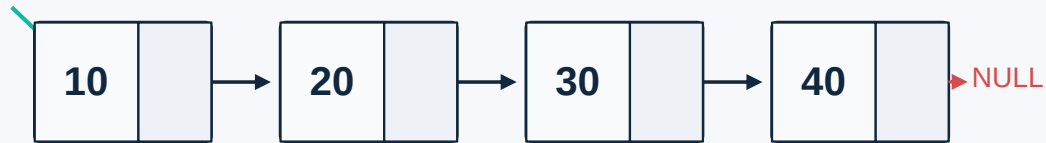
DO NOT

Do not call free(head) before moving HEAD to the second node — you would lose the next address.

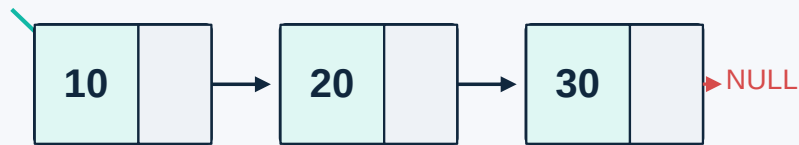
Deleting the last node

You need both the last node and the node immediately before it.

Before HEAD



After HEAD



- 1 Handle empty list
- 2 Handle one-node list separately
- 3 Move prev and ptr until ptr is last
- 4 prev->next = NULL
- 5 free(ptr)

WHY PREV?

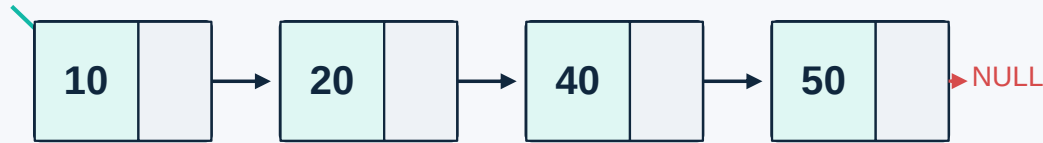
In a singly list, the last node cannot move backward to its predecessor.

Deleting the node after key = 20

Before HEAD



After HEAD



- 1 Find ptr where ptr->data == 20
- 2 Check ptr->next != NULL
- 3 temp = ptr->next
- 4 ptr->next = temp->next
- 5 free(temp)

EDGE CASE

If key is the last node, there is no node after it to delete.

Singly linked list: complexity snapshot

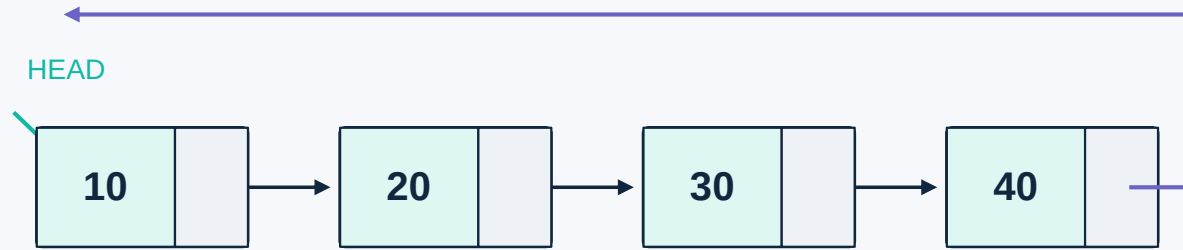
Operation	Time	Reason / note
Access by index	$O(n)$	Follow NEXT repeatedly
Search	$O(n)$	May inspect every node
Insert at beginning	$O(1)$	No traversal
Insert at end	$O(n)^*$	* $O(1)$ if a tail pointer is maintained
Delete first	$O(1)$	Advance HEAD
Delete last	$O(n)$	Find predecessor of last node

EXAM TIP

Know what makes an operation $O(1)$ versus $O(n)$: the deciding factor is usually whether traversal is needed.

Circular singly linked list

The last node points back to HEAD instead of storing NULL.



Termination condition

Stop when ptr returns to HEAD.

No natural end

There is no NULL link in a non-empty circular list.

Useful for

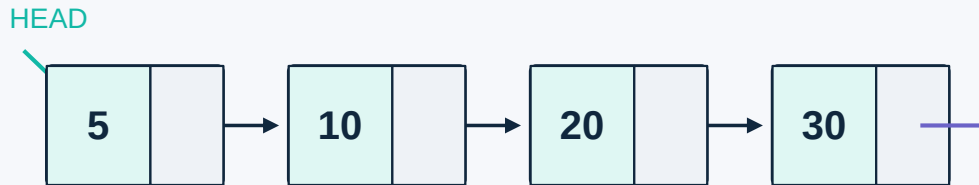
Round-robin scheduling, cyclic buffers and repeating sequences.

Circular list: insert at beginning

Before



After



1

If empty: $\text{new} \rightarrow \text{next} = \text{new}$; $\text{head} = \text{new}$

2

Else find last node

3

$\text{new} \rightarrow \text{next} = \text{head}$

4

$\text{last} \rightarrow \text{next} = \text{new}$

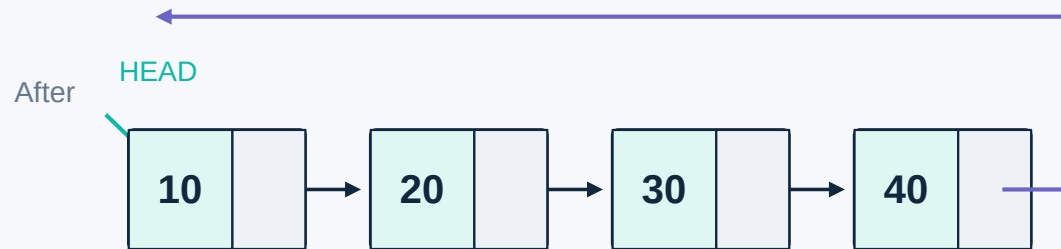
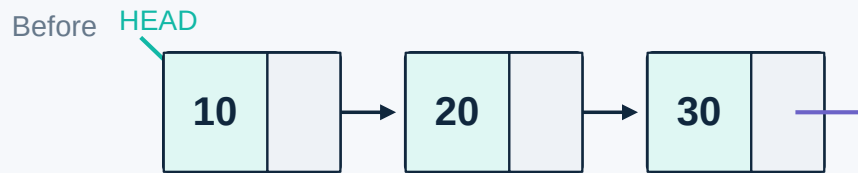
5

$\text{head} = \text{new}$

TIP

Maintaining a tail pointer avoids scanning for the last node.

Circular list: insert at end

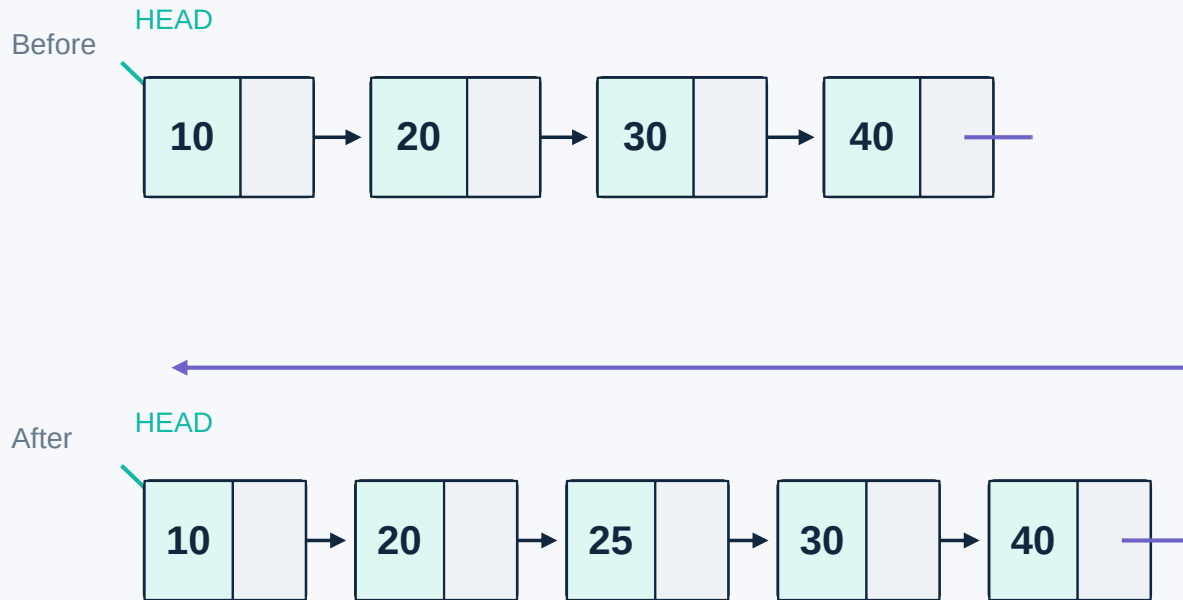


- 1 If empty: new points to itself
- 2 Otherwise find last node
- 3 $\text{new} \rightarrow \text{next} = \text{head}$
- 4 $\text{last} \rightarrow \text{next} = \text{new}$

POINTER CHECK

After insertion: exactly one link must point from the new last node back to HEAD.

Circular list: insert after key = 20



1

Search until key found or ptr returns to head

2

 $\text{new} \rightarrow \text{next} = \text{ptr} \rightarrow \text{next}$

3

 $\text{ptr} \rightarrow \text{next} = \text{new}$

SEARCH CONDITION

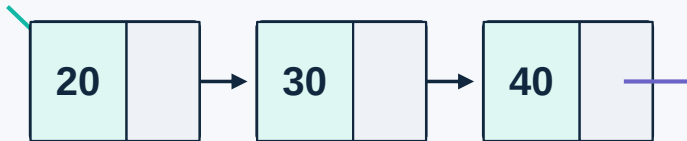
In a circular list, “not found” means one full cycle has been completed.

Circular list: delete first node

Before HEAD



After HEAD



1

Handle empty list

2

Handle one-node list

3

Find last node

4

`last->next = head->next`

5

`temp = head; head = head->next; free(temp)`

Circular list: delete last node

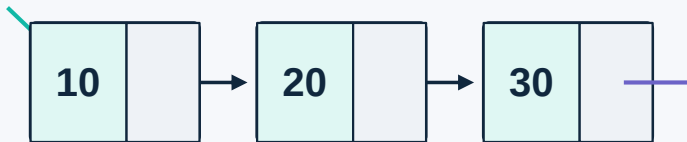
Before

HEAD



After

HEAD



1

Handle empty / one-node cases

2

Move prev and ptr until `ptr->next == head`

3

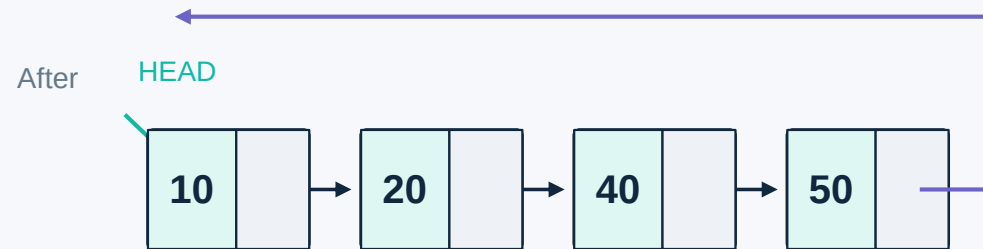
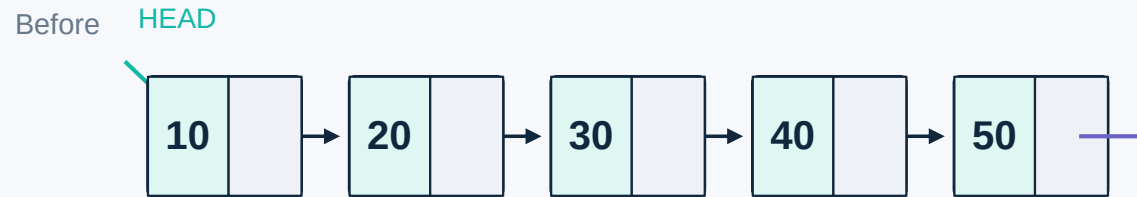
`prev->next = head`

4

`free(ptr)`**INVARIANT**

The new last node must point to HEAD — never to NULL.

Circular list: delete node after key = 20



- 1 Search for key node ptr
- 2 `temp = ptr->next`
- 3 If `temp == head`, deletion changes HEAD
- 4 `ptr->next = temp->next`
- 5 `free(temp)`

Circular linked list: common pitfalls

❶ Wrong stop condition

Using `ptr != NULL` causes an infinite loop in a valid non-empty circular list.

❷ Broken last link

After changing `HEAD`, the last node may still point to the old head.

❸ One-node case

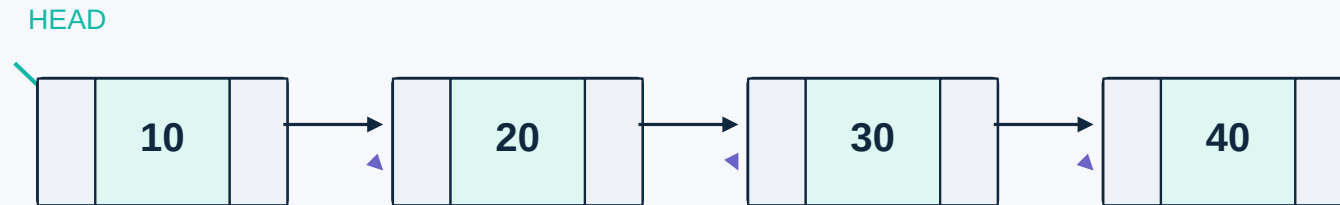
Deleting the only node must set `HEAD = NULL`.

Safe circular traversal

```
struct node *ptr = head;
if (ptr != NULL) {
    do {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    } while (ptr != head);
}
```

Doubly linked list

Each node can move both forward and backward.



PREV

Address of the preceding node.

DATA

Value stored in the node.

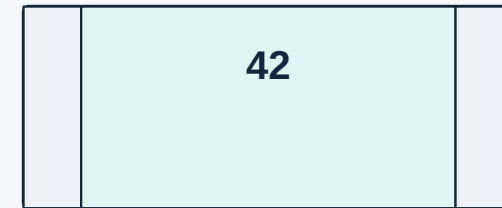
NEXT

Address of the succeeding node.

Doubly linked node in C

Node definition

```
struct node {  
    struct node *prev; // previous node  
    int data;          // stored value  
    struct node *next; // next node  
};
```



FIRST NODE

`head->prev == NULL`

LAST NODE

`last->next == NULL`

Doubly list: insert at beginning

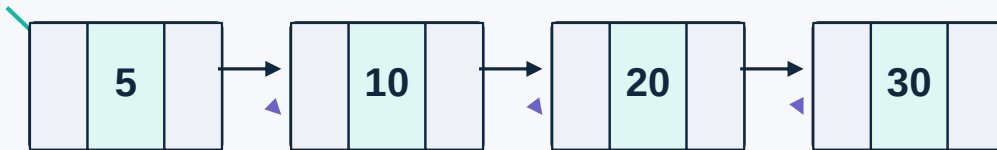
Before

HEAD



After

HEAD



1

`new->prev = NULL`

2

`new->next = head`

3

If `head != NULL`: `head->prev = new`

4

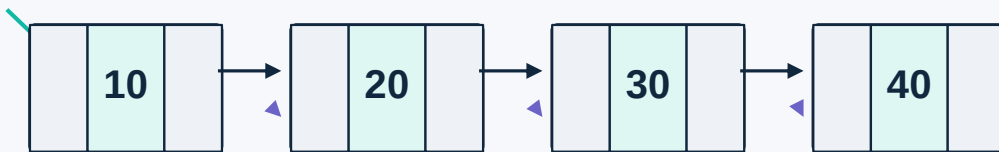
`head = new`

Doubly list: insert at end

Before HEAD



After HEAD



1

`new->next = NULL`

2

If empty: `new->prev = NULL`; `head = new`

3

Otherwise find last

4

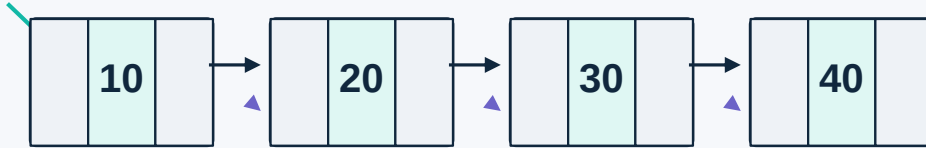
`last->next = new`

5

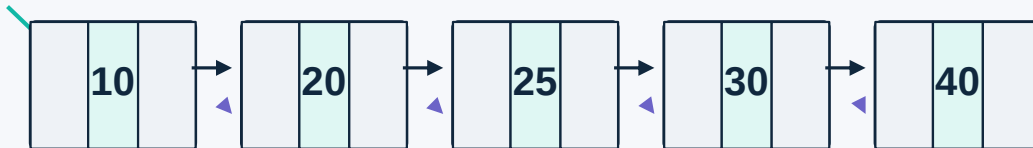
`new->prev = last`

Doubly list: insert after key = 20

Before HEAD



After HEAD



1

Find ptr with key 20

2

`new->next = ptr->next`

3

`new->prev = ptr`

4

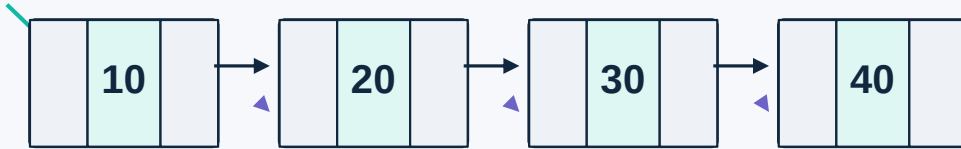
If successor exists: `successor->prev = new`

5

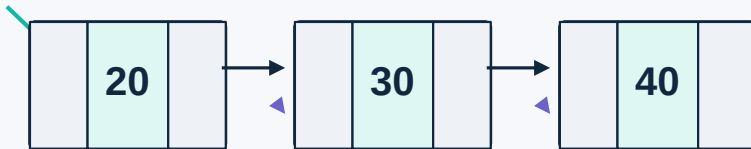
`ptr->next = new`

Doubly list: delete first node

Before HEAD

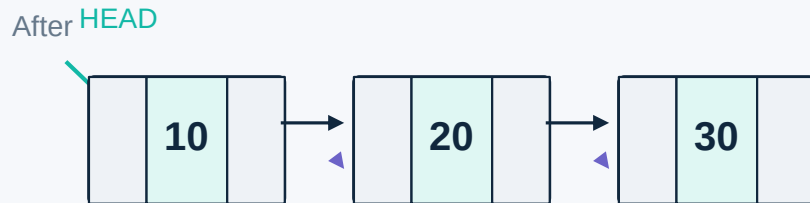
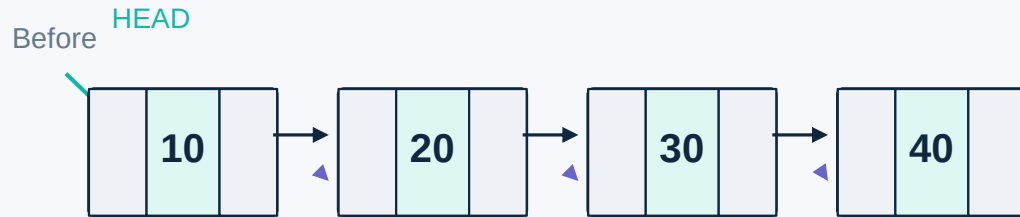


After HEAD



- 1 If empty $\rightarrow$ underflow
- 2 `temp = head`
- 3 `head = head->next`
- 4 If `head != NULL`: `head->prev = NULL`
- 5 `free(temp)`

Doubly list: delete last node



1

Find last node ptr (or use tail)

2

If $\text{ptr} \rightarrow \text{prev} == \text{NULL}$: list becomes empty

3

Else $\text{ptr} \rightarrow \text{prev} \rightarrow \text{next} = \text{NULL}$

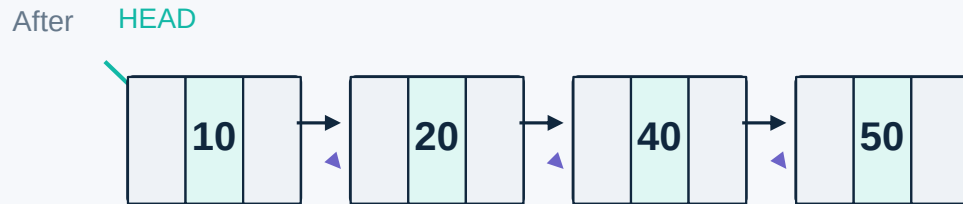
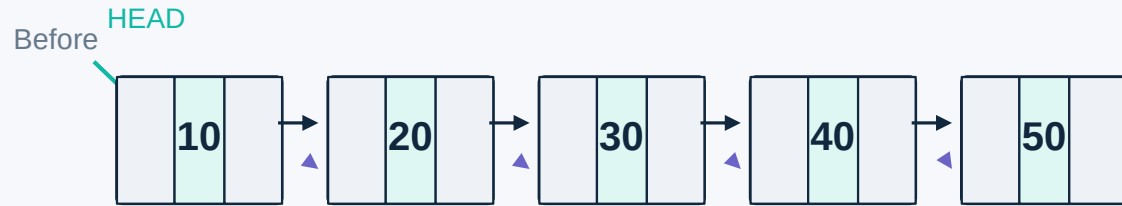
4

`free(ptr)`

ADVANTAGE

Once the last node is known, its predecessor is available directly through PREV.

Doubly list: delete node after key = 20



- 1 Find ptr with key 20
- 2 `temp = ptr->next`
- 3 `ptr->next = temp->next`
- 4 If `temp->next != NULL`: `temp->next->prev = ptr`
- 5 `free(temp)`

When is a doubly linked list worth it?

Better navigation

Move forward with NEXT and backward with PREV.

Simpler local deletion

Given a node pointer, its predecessor is available directly.

Higher memory cost

Every node stores one extra pointer and updates become more error-prone.

GOOD USE CASES

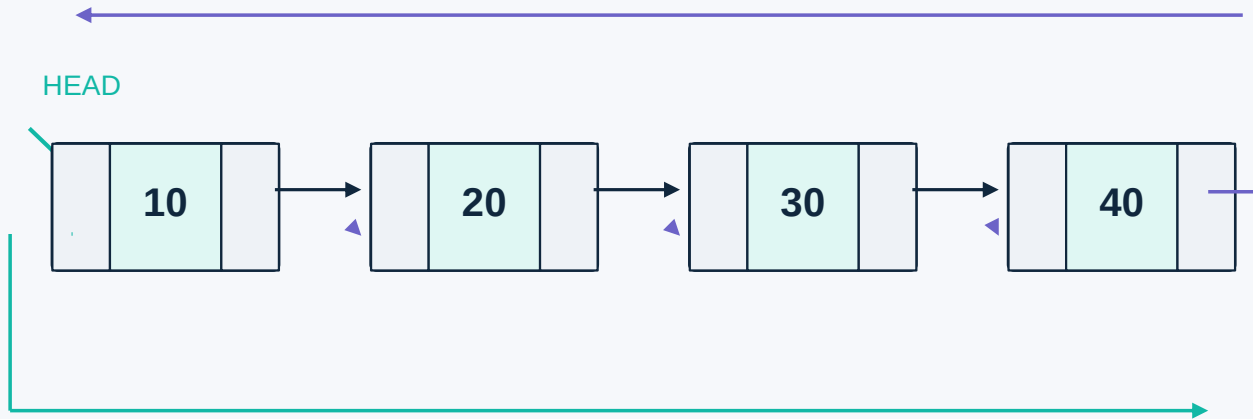
Browser back/forward history • Undo/redo • Deques • Navigable playlists • LRU cache lists

RULE OF THUMB

Use doubly linked lists when backward movement or deletion around a known node matters enough to justify the extra pointer.

Circular doubly linked list

Both directions wrap around — there are no NULL links in a non-empty list.



Last NEXT

Points to HEAD.

HEAD PREV

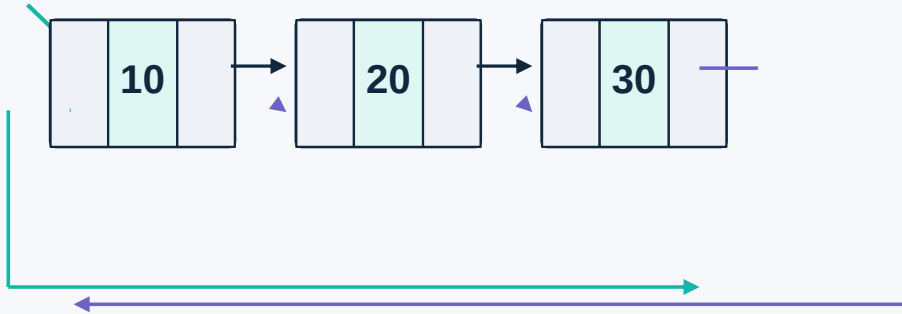
Points to the last node.

Navigation

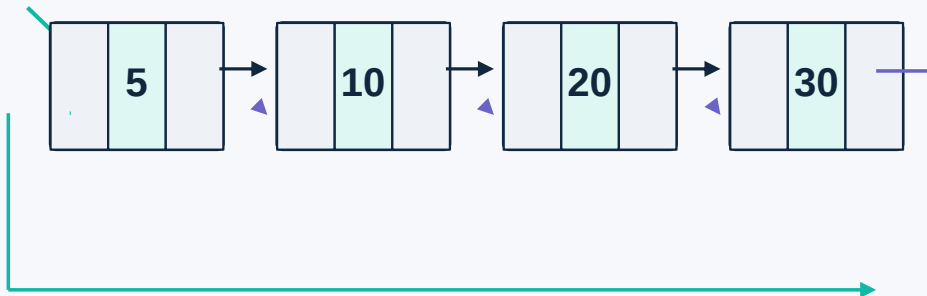
Move indefinitely in either direction.

Circular doubly list: insert at beginning

Before HEAD



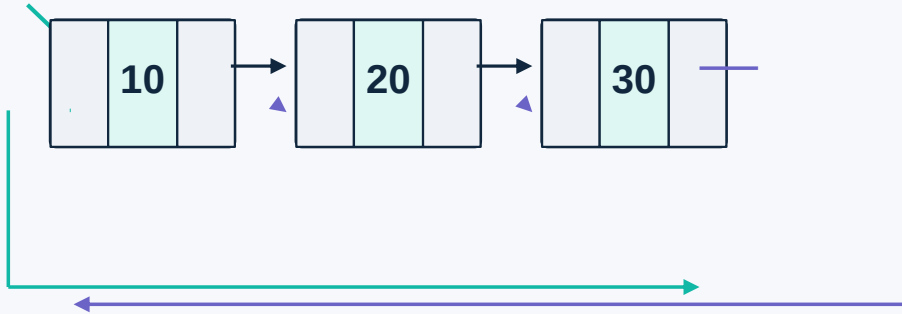
After HEAD



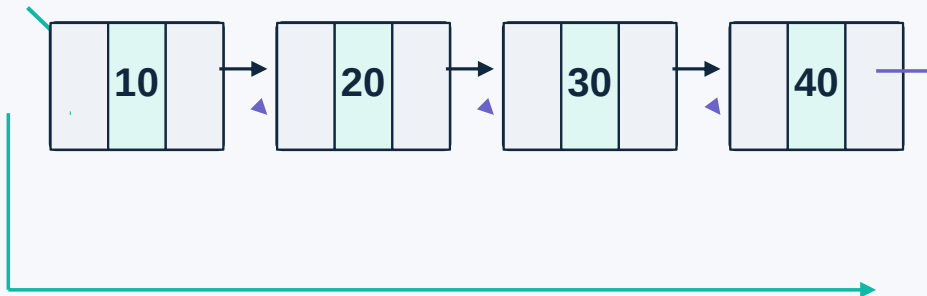
- 1 If empty: $\text{new} \rightarrow \text{next} = \text{new}$; $\text{new} \rightarrow \text{prev} = \text{new}$; $\text{head} = \text{new}$
- 2 $\text{last} = \text{head} \rightarrow \text{prev}$
- 3 $\text{new} \rightarrow \text{next} = \text{head}$; $\text{new} \rightarrow \text{prev} = \text{last}$
- 4 $\text{last} \rightarrow \text{next} = \text{new}$; $\text{head} \rightarrow \text{prev} = \text{new}$
- 5 $\text{head} = \text{new}$

Circular doubly list: insert at end

Before HEAD



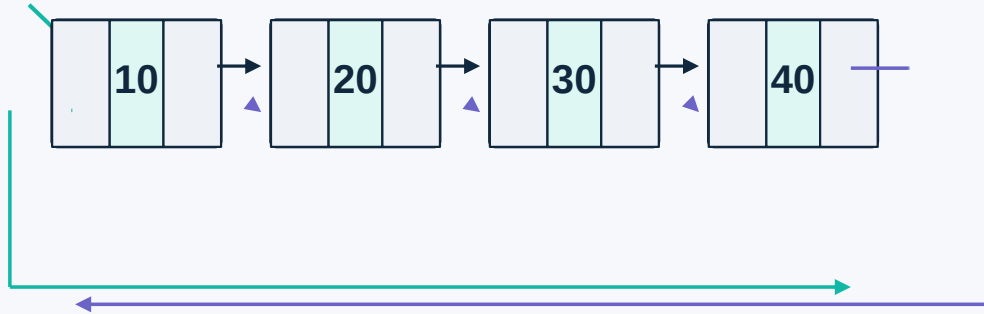
After HEAD



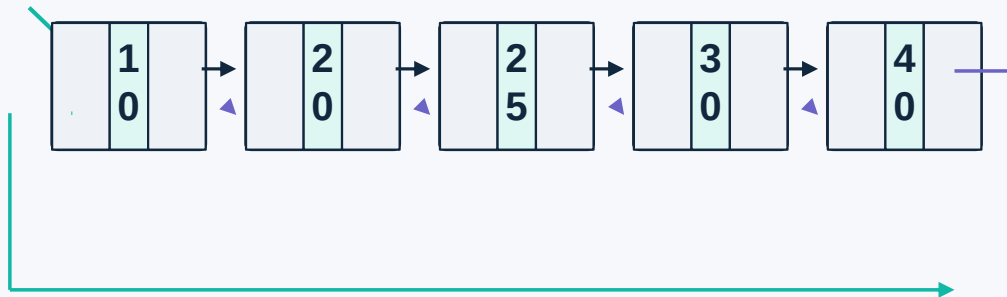
- 1 If empty: self-link and set head
- 2 $\text{last} = \text{head} \rightarrow \text{prev}$
- 3 $\text{new} \rightarrow \text{next} = \text{head}$
- 4 $\text{new} \rightarrow \text{prev} = \text{last}$
- 5 $\text{last} \rightarrow \text{next} = \text{new}$
- 6 $\text{head} \rightarrow \text{prev} = \text{new}$

Circular doubly list: insert after key = 20

Before HEAD



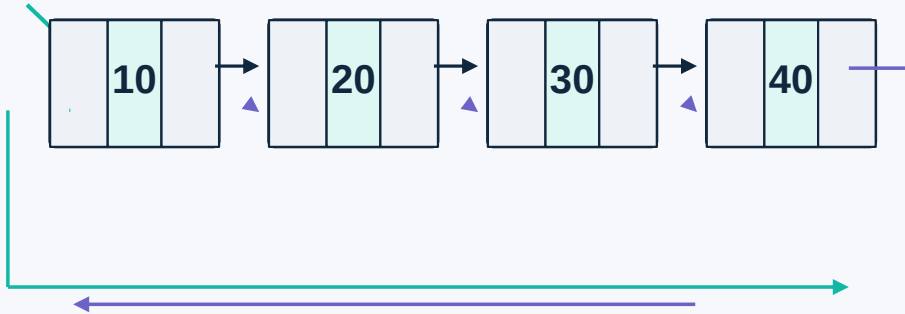
After HEAD



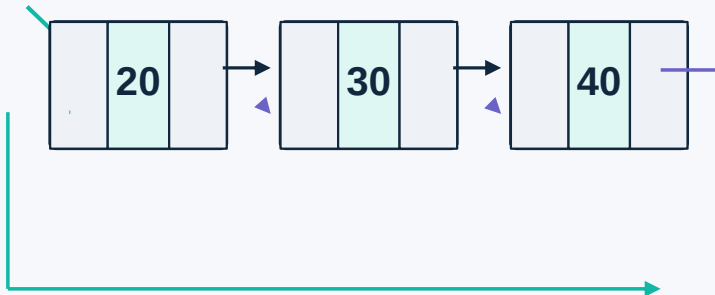
- 1 Find ptr containing key
- 2 `new->next = ptr->next`
- 3 `new->prev = ptr`
- 4 `ptr->next->prev = new`
- 5 `ptr->next = new`

Circular doubly list: delete first node

Before HEAD



After HEAD



1

If empty $\rightarrow$ underflow

2

If one node: `free(head); head = NULL`

3

`last = head->prev; temp = head`

4

`head = head->next`

5

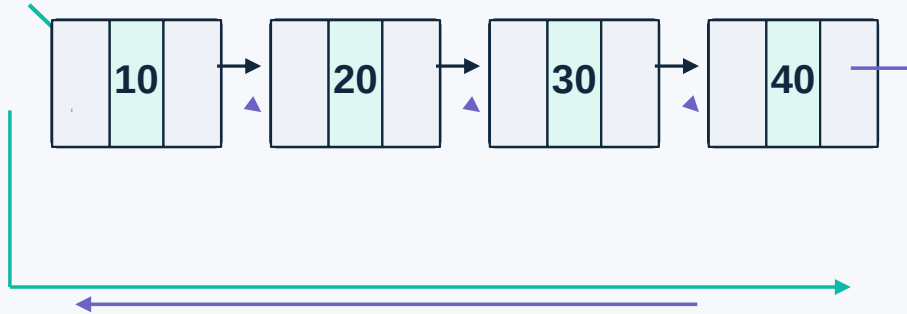
`head->prev = last; last->next = head`

6

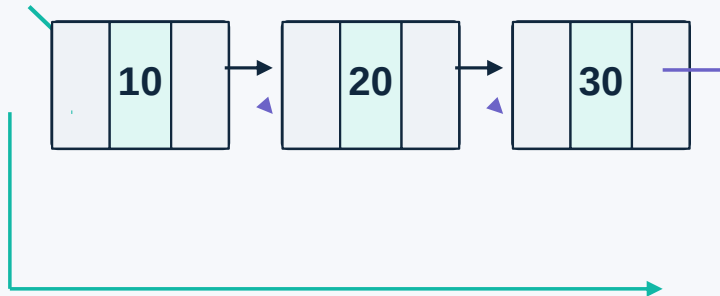
`free(temp)`

Circular doubly list: delete last node

Before HEAD

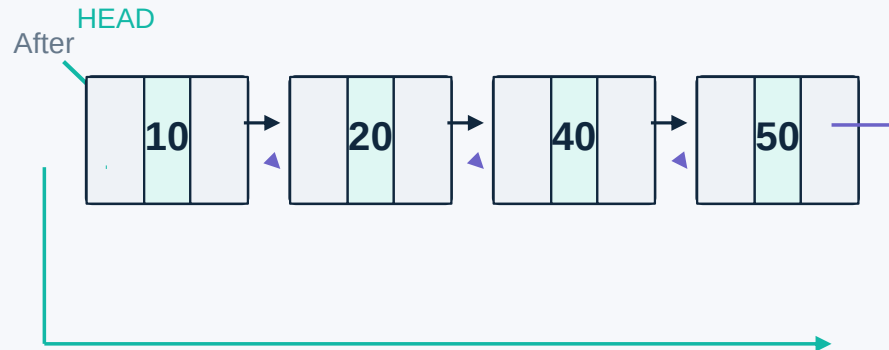
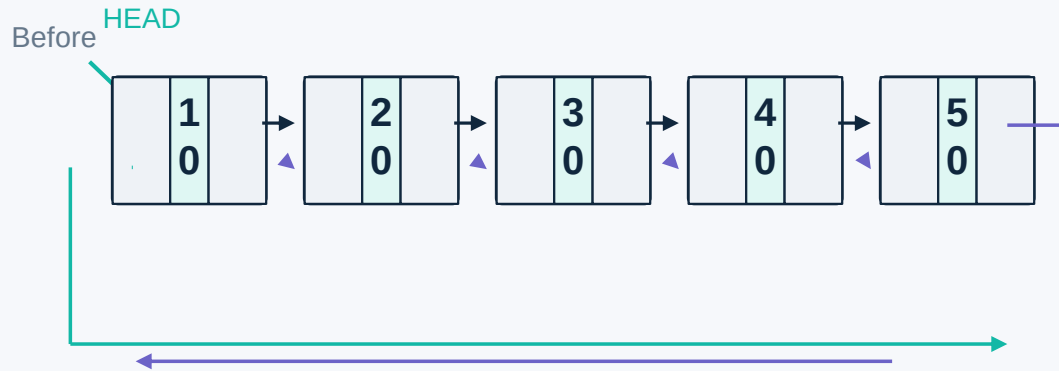


After HEAD



- 1 If empty $\rightarrow$ underflow
- 2 If one node: free and set head = NULL
- 3 $\text{temp} = \text{head} \rightarrow \text{prev}$
- 4 $\text{newLast} = \text{temp} \rightarrow \text{prev}$
- 5 $\text{newLast} \rightarrow \text{next} = \text{head}$
- 6 $\text{head} \rightarrow \text{prev} = \text{newLast}$
- 7 $\text{free}(\text{temp})$

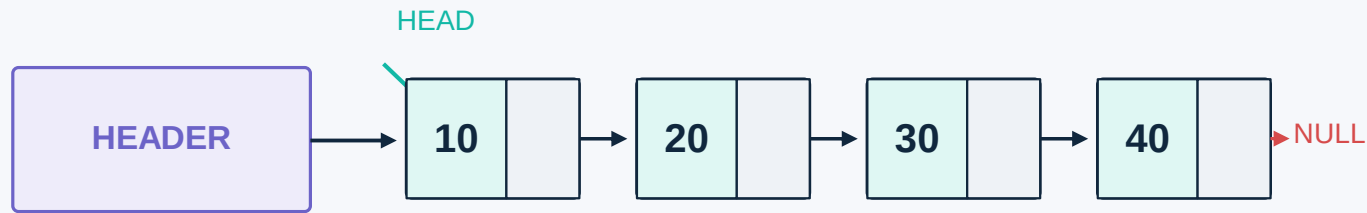
Circular doubly list: delete node after key = 20



- 1 Find ptr containing key
- 2 `temp = ptr->next`
- 3 If `temp == head`: deletion changes HEAD
- 4 `ptr->next = temp->next`
- 5 `temp->next->prev = ptr`
- 6 `free(temp)`

Header linked lists

A permanent header node simplifies some edge cases and can store metadata.



Grounded header list

Last node stores NULL. START points to HEADER, not directly to the first data node.

Circular header list

Last node points back to HEADER. The header acts as a natural sentinel/end marker.

Circular header list: traversal & insertion

Key pointer pattern

```
// Traverse
ptr = head->next;
while (ptr != head) {
    printf("%d ", ptr->data);
    ptr = ptr->next;
}

// Insert after ptr
new->next = ptr->next;
ptr->next = new;
```

1

Start at head->next, the first data node

2

Process until the cursor returns to header

3

For insertion after a known node, preserve its successor first

4

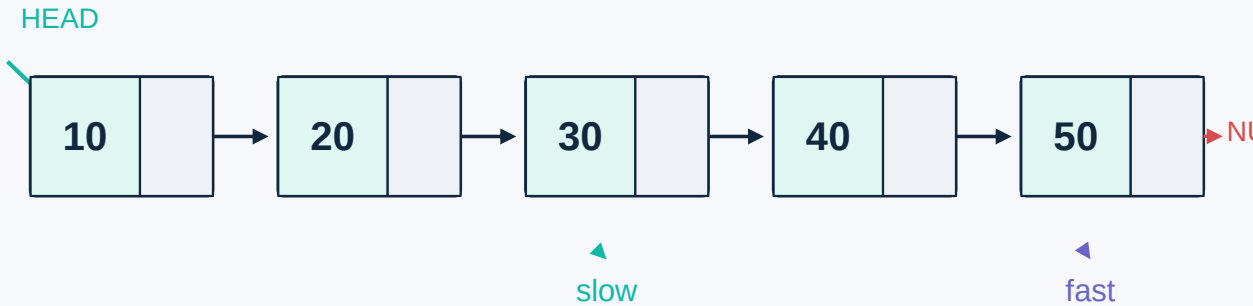
Redirect ptr->next to the new node

BENEFIT

The header node gives a fixed sentinel address that can simplify termination tests.

Practice 1: find the middle node

Use one slow pointer and one fast pointer.



Slow-fast pointer technique

```
slow = head;
fast = head;

while (fast != NULL && fast->next != NULL) {
    slow = slow->next;          // moves 1 node
    fast = fast->next->next;    // moves 2 nodes
}

printf("Middle = %d", slow->data);
```

WHY IT WORKS

When FAST reaches the end, SLOW has covered about half the distance.

Practice 2: create n nodes and display them

1

Read n

2

Repeat n times: allocate and fill a node

3

If first node: head = tail = new

4

Else: tail->next = new; tail = new

5

Traverse from head and print

Efficient creation using tail

```
struct node *head = NULL, *tail = NULL;

for (int i = 0; i < n; i++) {
    struct node *new = malloc(sizeof *new);
    scanf("%d", &new->data);
    new->next = NULL;

    if (head == NULL) head = tail = new;
    else { tail->next = new; tail = new; }
}

display(head);
```

IMPROVEMENT

Using a tail pointer avoids traversing the list again for every new node; building n nodes becomes $O(n)$, not $O(n^2)$.

Compare linked-list variants

Choose the structure that matches the operations you need most often.

Operation	Singly	Circular	Doubly	C. Doubly	Why
Pointer(s) / node	1	1	2	2	Memory
Natural end	NULL	HEAD	NULL	HEAD	Traversal stop
Backward traversal	No	No	Yes	Yes	Navigation
Insert front	$O(1)$	$O(n)^*$	$O(1)$	$O(1)$	$^*O(1)$ with tail
Delete last	$O(n)$	$O(n)^*$	$O(n)^{**}$	$O(1)$	$^{**}O(1)$ with tail
Best fit	Simple	Cyclic	Bidirectional	Cyclic + bi	Use case

Common C mistakes — and how to prevent them

NULL dereference

Always test a pointer before using `->` on it.

Lost sub-list

Save the old link before overwriting a pointer.

Memory leak

`free()` nodes that are removed and no longer reachable.

Use-after-free

Never access a node after `free(temp)`.

Missed edge case

Test: empty list, one node, first node, last node.

Infinite circular loop

Stop after a full cycle — not at NULL.

DEBUGGING HABIT

After every update, redraw the links and verify the structure invariants before running the next line.

Quick check: can you reason about the links?

- 1 Why must `new->next = head` happen before `head = new`?
- 2 What condition ends traversal in a circular list?
- 3 Why is deleting the last node easier in a doubly list?
- 4 Which operation becomes $O(1)$ when a tail pointer is maintained?
- 5 What does `free(ptr)` do that `ptr = NULL` does not?

EXIT TICKET

Explain one linked-list operation using only a diagram and pointer assignments.